\# MLOps for Heart Disease Classification

Course: AIMLCZG523 — Machine Learning Operations Assignment: 01 Student: Sarth Nitin Khandekar

Student ID: 2025AF05161 Date: 20 October 2026

1\. Abstract

This project implements an educational binary classification workflow using the processed Cleveland subset of the UCI Heart Disease dataset. The workflow covers data acquisition, exploratory analysis, preprocessing, model comparison, experiment tracking, automated tests, containerization, API serving, and deployment configuration. The model is not clinically validated and is not intended for diagnosis.

2\. Problem statement and objectives

The goal is to demonstrate reproducible ML engineering rather than build a medical device. The target is transformed to binary: original target 0 becomes absence; values 1–4 become presence. Objectives include repeatable data preparation, fair model evaluation, experiment traceability, tested inference, and a deployable service.

3\. Dataset and acquisition

Source: UCI Machine Learning Repository, Heart Disease dataset: https://archive.ics.uci.edu/dataset/45/heart+disease. The downloader retrieves the archive and extracts processed.cleveland.data. There are 13 input features and one target column. ? values are represented as missing and imputed within the model pipeline.

4\. EDA

Exploratory Data Analysis (EDA) was performed to understand the dataset's structure, feature distributions, missing values, and relationships with the target variable.

The analysis included checking dataset dimensions, data types, missing values, target-class distribution, and numerical feature statistics. These checks helped identify the data characteristics and prepare the dataset for model development.

5\. Feature engineering and models

The feature pipeline standardizes numeric measurements and imputes missing values. Coded clinical categories are imputed with the most frequent value and one-hot encoded. The preprocessing is fitted within each training fold to reduce data leakage. Models compared: class-weighted Logistic Regression and Random Forest. Data split: 80% train / 20% hold-out test, stratified with random seed 42. Five-fold stratified CV is run on the training partition. Final model selection rule: highest mean CV ROC-AUC.

6\. Evaluation

The models were evaluated using a stratified 80:20 train-test split with random seed 42. Five-fold stratified cross-validation was performed on the training partition, and the final evaluation metrics were calculated on the hold-out test set.

Logistic Regression was selected because it achieved the highest mean cross-validation ROC-AUC (0.9025), compared with 0.8975 for Random Forest. Although Random Forest achieved slightly higher hold-out accuracy and recall, model selection followed the predefined rule of choosing the model with the highest mean cross-validation ROC-AUC.

Results:

Model	Test accuracy	Test precision	Test recall	Test ROC-AUC	CV ROC-AUC mean

-	-:	-:	-:	-:	-:
Logistic Regression	0.8689	0.8125	0.9286	0.9665	0.9025
Random Forest	0.8852	0.8182	0.9643	0.9513	0.8975

\### Cross-Validation Results:

Model	CV Accuracy Mean	CV Accuracy Std. Dev.	CV Precision Mean	CV Recall Mean
-	-	-	-	-
Logistic Regression	0.8552	0.0238	0.8731	0.8099
Random Forest	0.8016	0.0169	0.7831	0.7917

Discussion:

On the hold-out test set, Logistic Regression achieved an accuracy of 86.89%, precision of 81.25%, recall of 92.86%, and ROC-AUC of 0.9665. Its high recall indicates that it identified a large proportion of positive cases in this particular test set. Random Forest achieved an accuracy of 88.52% and recall of 96.43%, but its mean cross-validation ROC-AUC was slightly lower than that of Logistic Regression.

The results demonstrate that model selection should consider cross-validation performance and multiple evaluation metrics rather than accuracy alone. However, the difference in cross-validation ROC-AUC between the models is small, and the results do not establish that one model will consistently outperform the other on new populations.

The dataset is relatively small, so the evaluation results may vary with different data splits. The model is intended for educational demonstration only. It has not undergone clinical validation and must not be used to diagnose heart disease or make medical decisions.

7\. Experiment tracking

MLflow was used to track model training experiments and record model parameters, evaluation metrics, and supporting artifacts. The experiment heart-disease-assignment contains runs for Logistic Regression and Random Forest.

Both models were evaluated using a stratified 80:20 train-test split and five-fold cross-validation on the training partition, with random seed 42. The logged metrics include accuracy, precision, recall, ROC-AUC, cross-validation means, and accuracy standard deviation.

Logistic Regression was selected because it achieved the higher mean cross-validation ROC-AUC of 0.9025, compared with 0.8975 for Random Forest. The finished Logistic Regression run ID is f78ee6b994174177b3f52dd11e7afb31, and the finished Random Forest run ID is 46f89ed1666b401f9e4ea7dc9ebada6e.

MLflow artifacts include the classification report, confusion matrix plot, and ROC curve for Logistic Regression. The tracking database is stored locally in mlflow.db, while run artifacts are stored under mlruns.

8\. Packaging and reproducibility

The selected Logistic Regression model is saved as a serialized scikit-learn pipeline at models/heart_disease_pipeline.joblib. The pipeline includes preprocessing and the trained classifier, allowing the same transformations to be applied during inference. The accompanying models/metadata.json records the selected model, input features, target mapping, random seed, test split, and evaluation metrics.

The dataset is split into 80% training data and 20% test data using random seed 42. Five-fold stratified cross-validation is performed on the training partition, and the model with the highest mean cross-validation ROC-AUC is selected. Preprocessing is included in the pipeline to help prevent data leakage between training and validation folds.

Project dependencies are specified in requirements.txt, while pyproject.toml provides project configuration and supports editable installation. The FastAPI service loads the saved model artifact from the configured MODEL_PATH, or from the default path models/heart_disease_pipeline.joblib.

The project can be run in a Python virtual environment by installing the declared dependencies and installing the package in editable mode. Dataset acquisition and model training are automated through Python modules, and the same workflow is exercised by the GitHub Actions CI pipeline.

The serialized model, metadata, and dependency definitions support reproducibility. However, exact reproduction may also depend on the Python version, dependency versions, dataset availability, and execution environment.

9\. API, Docker and monitoring

FastAPI exposes /health and /predict. The /health endpoint returned HTTP 200 OK during testing, confirming that the service was reachable and the expected model artifact path existed. The /predict endpoint also returned HTTP 200 OK through Kubernetes port forwarding and produced a prediction, class probabilities, confidence score, and educational-use disclaimer. The response includes the X-Process-Time-ms header, and request middleware logs the request method, path, status code, and processing duration.

The API is packaged in a Docker image and deployed to a local Kubernetes cluster. The model is intended for educational purposes only and has not been clinically validated. The reported confidence is a model probability and should not be interpreted as calibrated clinical certainty.

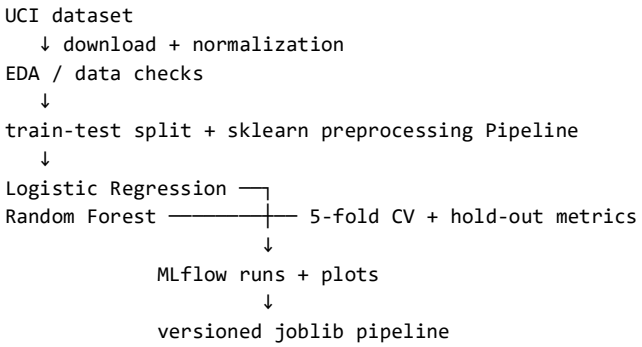
10\. CI/CD, deployment, and conclusion

GitHub Actions automates code-quality checks with Ruff, runs the pytest suite, and performs dataset acquisition and a model-training smoke test. All seven project tests passed during local testing, and the CI workflow was configured to run these checks on GitHub.

The application was deployed to a local Kubernetes cluster using a Deployment and NodePort Service. The Deployment reported one available replica, and the running pod had zero restarts at the time of verification. The /health endpoint returned HTTP 200 OK, and a POST request to /predict also returned HTTP 200 OK through Kubernetes port forwarding. The tested local port-forward URL was http://localhost:18001; this forwarding session is temporary and must be started again when needed.

This project demonstrates a reproducible machine-learning workflow covering data acquisition, exploratory analysis, model training, MLflow experiment tracking, API packaging, automated testing, containerization, Kubernetes deployment, and request logging. Limitations include the small dataset, local-only deployment, lack of production-grade drift monitoring, and absence of clinical validation. Future work could include stronger data validation, model registry and staging, security improvements, drift monitoring, and a defined retraining policy. The model must not be used for medical decisions.

Architecture



```
                ↓
 FastAPI /predict + request logs
                ↓
      Docker container
                ↓
Kubernetes Deployment + Service
```

Evidence appendix

\- API health and prediction evidence — reports/evidence/Kubernetes - Deployment, Health_Check, Prediction.png

\- Kubernetes Deployment, pod, and Service evidence — reports/evidence/Kubernetes - Deployment, Health_Check, Prediction.png

\- GitHub repository: https://github.com/Sarth05/MLOps_Assignment01